

内存分区

全局区

```
#include<iostream>
using namespace std;

//全局区：全局变量（函数外定义的变量），全局常量（const），静态变量（static），字符串常量（"hello world"）
//栈区：局部变量（函数内定义的变量），局部常量（const），形参

const int a=10;

int main()
{
    const int b=10;
    static int c=10;

    cout<<"hello world"<<endl;
    cout<<&a<<endl;
    cout<<&b<<endl;
    cout<<&c<<endl;
    return 0;
}
```

栈区

```
#include <iostream>
using namespace std;

//栈区的数据由编译器管理与开辟
int* function(int b)
{
    int a = 10;
    return &a; // 返回局部变量的地址，这是不安全的
}

int main()
{
    int *p = function(1); //执行完函数后堆区会被释放
    cout << *p << endl; //只保留一次
    cout << *p << endl; //第二次输出就是错误内容
    return 0;
}
```

堆区

```
#include<iostream>
using namespace std;

//堆区由程序员分配释放；若程序员不释放，程序结束时由操作系统回收
//利用new在堆区开辟内存（动态分配内存空间）
int* func()
{
    int *p=new int(10);
    return p;
}

int main()
{
    int *p=func();
    cout<<*p<<endl; //堆区内存不会被自动释放
    cout<<*p<<endl;
    cout<<*p<<endl;
    return 0;
}
```

new

```
#include<iostream>
using namespace std;

int* func()
{
    //new返回的是该数据类型的指针（地址），所以int *p=...
    int *p=new int(10);
    return p;
}

void func2()
{
    int *arr=new int[10]; //在堆区创建10整型数据的数组
    for(int i=0;i<10;i++)
    {
        arr[i]=i+100;
    }
    for(int i=0;i<10;i++)
    {
        cout<<arr[i]<<endl;
    }

    //释放堆区数组，释放数组时加上[]
    delete[] arr;
    for(int i=0;i<10;i++)
    {
        cout<<arr[i]<<endl;
    }
}
```

```
}

void func1()
{
    int *p=func();
    cout<<*p<<endl;
    cout<<*p<<endl;
    //如果想释放堆区的数据，利用delete
    delete p;
    cout<<*p<<endl; //内存已经被释放，非法操作
}

int main()
{
    func1();
    func2();
    return 0;
}
```